

ACTIVIDAD 06 DE 46 - VULNERABILIDADES EN DEPENDENCIAS: PIP-AUDIT Y CVES

Segunda hora de clase (50 minutos) | Trabajo practico asistido con IA

Herramienta principal: opencode (CLI). Alternativas: ChatGPT, Claude, Copilot, Gemini.

Base normativa / marco de referencia: Gestion de vulnerabilidades y dependencias (GR.4, GA.1 del Marco Agesic); CVE/CWE

1. Contexto

No solo tu código tiene vulnerabilidades: también las bibliotecas que usas. Hoy instalas pip-audit, una herramienta gratuita que compara tus dependencias con las bases de datos de CVEs, y aprenderás a leer un CVE, actualizar de forma segura y dejar evidencia del análisis.

2. Marco teorico

¿Que son las dependencias?

Son las bibliotecas y paquetes que tu código usa (ej. Flask, requests). Tu código es tan seguro como lo sean sus dependencias.

Cadena de suministro de software

Un atacante puede comprometer una biblioteca popular y con ella a todos los que la usan. Por eso se auditan las dependencias.

- CVE: identificador de una vulnerabilidad conocida.
- SBOM (Software Bill of Materials): lista de componentes de tu software, como un "ingredientes" de seguridad.

pip-audit

pip-audit compara las dependencias instaladas contra bases de datos de vulnerabilidades conocidas (CVE) y avisa de las afectadas.

Mitigaciones

- Actualizar dependencias regularmente.
- Congelar versiones y validar los cambios.
- Usar entornos virtuales aislados.
- Escanear en cada integración (CI).

3. Objetivos de aprendizaje

- Instalar pip-audit y auditar las dependencias del proyecto.
- Interpretar un CVE: descripcion, severidad (CVSS) y versiones afectadas.
- Actualizar dependencias de forma segura y re-auditar.
- Documentar la gestion de vulnerabilidades.

4. Herramientas del dia (instalacion paso a paso)

pip-audit (auditor gratuito de dependencias)

Revisa las dependencias de Python y las cruza con la base de datos de vulnerabilidades conocidas.

1. Activa el entorno virtual del proyecto de la Actividad 4.
2. Instala: ``pip install pip-audit``.
3. Verifica: ``pip-audit --version``.

5. Desarrollo paso a paso

Paso 1: Auditar el proyecto

Ejecuta ``pip-audit`` dentro del entorno virtual y observa los CVEs reportados.

Paso 2: Interpretar un CVE

Pide a la IA que explique el CVE mas grave encontrado (seccion 5).

Paso 3: Planear la actualizacion

Verifica que la actualizacion no rompa la API (lee notas de version).

Paso 4: Actualizar

Actualiza la dependencia: ``pip install --upgrade <paquete>`` y vuelve a ejecutar la API.

Paso 5: Re-auditar

Ejecuta de nuevo ``pip-audit`` y confirma que el hallazgo desaparecio o cambio.

Paso 6: Generar reporte

Genera el reporte en JSON: ``pip-audit --format json -o evidencias/audit.json``.

Paso 7: Capturar evidencias

Guarda capturas del antes y el despues.

Paso 8: Documentar y reflexionar

Escribe el informe y tu reflexion.

6. Prompts sugeridos para la IA (opencode)

Copia y adapta estos mensajes en opencode (comando interactivo o `opencode` en terminal).

Explicame este CVE encontrado en mi proyecto: [pega el hallazgo]. ¿Que es CVSS, que significa la severidad y como afectaria a una institucion financiera?

Dame un procedimiento de gestion de dependencias para un banco: con que frecuencia auditar, quien decide actualizar y como probar los cambios.

¿Por que actualizar una dependencia puede romper la aplicacion? ¿Como minimizar ese riesgo?

7. Documentacion del RSI (bitacora)

Registra en `bitacora.md` de la carpeta de la actividad:

- Resultado de la auditoria inicial.
- CVE interpretado (severidad y versiones).
- Decision de actualizacion y resultado.
- Politica propuesta de revision de dependencias.

8. Evidencias a capturar y guardar

Guarda cada evidencia en la carpeta `evidencias/` con el nombre EVI-YYYY-MM-DD-NN-descripcion.ext y registrala en la bitacora.

- EVI-...-01-pipaudit-antes.txt (auditoria inicial).
- EVI-...-02-pipaudit-despues.txt (tras actualizar).
- EVI-...-03-audit.json (reporte estructurado).
- EVI-...-04-cve-explicado.png (explicacion con IA).

9. Informe segun el marco de referencia

Elabora el informe de la actividad (archivo `informe.md`) que incluya:

- Metodologia de la auditoria.
- CVEs encontrados y su severidad.
- Acciones tomadas (actualizaciones) y verificacion.
- Politica propuesta para el futuro.

10. Entregable de la clase

Auditoria de dependencias antes/despues + reporte JSON + politica de gestion propuesta.

Anexo A - Informe completo para presentar al BCU

Plantilla del informe ejecutivo que el RSI entregaria al Banco Central del Uruguay (BCU) por la actividad 06 - Vulnerabilidades en dependencias: pip-audit y CVEs. Completa los campos [entre corchetes] con los resultados reales de la actividad y adjunta las evidencias EVI-...

Encabezado del informe

Institucion: Banco Ficticio del Uruguay

Responsable de Seguridad de la Informacion (RSI): [Nombre y cargo]

Destinatario: Banco Central del Uruguay - Superintendencia de Instituciones Financieras

Asunto: Actividad 06 - Vulnerabilidades en dependencias: pip-audit y CVEs

Fecha: [dd/mm/aaaa] | Version: 1.0 | Clasificacion: Confidencial

1. Resumen ejecutivo

[2 a 5 parrafos breves: que se realizo, en que marco normativo, que se encontro, que se corrigio y el estado actual. Debe poder leerse en 1 minuto por un directivo.]

2. Marco normativo y regulatorio aplicable

- Norma de Seguridad de la Informacion y Ciberseguridad del BCU (SF.SEG.08).
- Recopilacion de Normas de Regulacion y Control del Sistema Financiero (RNRCSF).
- Circular sobre gestion de incidentes y notificacion de eventos de ciberseguridad.
- Normativa interna del Banco Ficticio del Uruguay: [politica de seguridad, plan de trabajo RSI].

3. Alcance y metodologia

- Alcance: [activos, procesos, sistemas o areas cubiertos por la actividad].
- Metodo: [pruebas tecnicas, revision documental, simulacion, escaneo, etc.].
- Herramientas utilizadas: pip-audit (auditor gratuito de dependencias).
- Periodo de ejecucion: [fecha de inicio] a [fecha de fin].

4. Descripcion de la actividad

No solo tu codigo tiene vulnerabilidades: tambien las bibliotecas que usas. Hoy instalas pip-audit, una herramienta gratuita que compara tus dependencias con las bases de datos de CVEs, y aprenderas a leer un CVE, actualizar de forma segura y dejar evidencia del analisis.

5. Resultados obtenidos

- Objetivo: Instalar pip-audit y auditar las dependencias del proyecto. -> [Logrado / Parcial / No logrado]
- Objetivo: Interpretar un CVE: descripcion, severidad (CVSS) y versiones afectadas. -> [Logrado / Parcial / No logrado]

- Objetivo: Actualizar dependencias de forma segura y re-auditar. -> [Logrado / Parcial / No logrado]
- Objetivo: Documentar la gestion de vulnerabilidades. -> [Logrado / Parcial / No logrado]

6. Riesgos identificados y evaluacion

Tabla de riesgos detectados con impacto en la institucion financiera:

ID Riesgo	Descripcion	Probabilidad	Impacto	Nivel	Tratamiento
R-01	[riesgo encontrado]	[A/M/B]	[A/M/B]	[C/E/M]	[Mitigado / Aceptado / En plan]
R-02	[riesgo encontrado]	[A/M/B]	[A/M/B]	[C/E/M]	[Mitigado / Aceptado / En plan]

7. Controles implementados

Descripcion de los controles aplicados y su estado:

- Control: Instala y ejecuta pip-audit -> [Implementado / Parcial / Pendiente]
- Control: Interpreta CVEs y severidad (CVSS) -> [Implementado / Parcial / Pendiente]
- Control: Actualiza de forma segura y re-audita -> [Implementado / Parcial / Pendiente]
- Control: Genera y guarda el reporte JSON -> [Implementado / Parcial / Pendiente]
- Control: Propone una politica de gestion de dependencias -> [Implementado / Parcial / Pendiente]

8. Incidentes de seguridad relacionados

[Ninguno en esta actividad. O bien: fecha, tipo, clasificacion, datos afectados, impacto y medidas adoptadas. Si corresponde, nota que ya se notifico al BCU en los plazos reglamentarios.]

9. Indicadores y metricas de la actividad

- Tiempo de ejecucion: [X] horas.
- Hallazgos: [X] altos, [X] medios, [X] bajos.
- Hallazgos corregidos: [X] / [X].
- Cobertura: [X]% de los activos del alcance.

10. Plan de accion y proximos pasos

Accion	Responsable	Plazo	Estado
[accion pendiente]	[cargo]	[dd/mm/aaaa]	Pendiente

11. Conclusion

[Conclusion ejecutiva: postura de seguridad de la institucion tras la actividad y compromiso de seguimiento.]

12. Firma y aprobacion

Firma del RSI: _____ Fecha: [dd/mm/aaaa]

Visto bueno del area de negocio / Direccion: _____

Anexos: evidencias EVI-... registradas en la bitacora.

Anexo B - Como presentar la actividad a AGESIC

Guia de estructura y contenido para comunicar la actividad 06 - Vulnerabilidades en dependencias: pip-audit y CVEs a la Agencia para el Desarrollo del Gobierno de Gestion Electronica y la Sociedad de la Informacion (AGESIC), alineada al marco de ciberseguridad de referencia para Uruguay.

Encuadre institucional

- Organismo: Banco Ficticio del Uruguay, dependencia publica.
- RSI: [Nombre y cargo] | Correo: [correo institucional].
- Fecha de presentacion: [dd/mm/aaaa] | Periodo cubierto: [periodo].

1. Presentacion del responsable

Quien es el RSI, su rol, a que direccion reporta y como se vincula con la gestion de la seguridad de la informacion en el Banco Ficticio del Uruguay.

2. Objetivo de la actividad

No solo tu codigo tiene vulnerabilidades: tambien las bibliotecas que usas. Hoy instalas pip-audit, una herramienta gratuita que compara tus dependencias con las bases de datos de CVEs, y aprenderas a leer un CVE, actualizar de forma segura y dejar evidencia del analisis.

- Objetivo: Instalar pip-audit y auditar las dependencias del proyecto.
- Objetivo: Interpretar un CVE: descripcion, severidad (CVSS) y versiones afectadas.
- Objetivo: Actualizar dependencias de forma segura y re-auditar.
- Objetivo: Documentar la gestion de vulnerabilidades.

3. Marco de referencia utilizado

- Marco normativo de ciberseguridad aplicable (AGESIC / normativa nacional).
- Ley 18.331 de Proteccion de Datos Personales y su Decreto Reglamentario.
- Guia o documento de base: Gestion de vulnerabilidades y dependencias (GR.4, GA.1 del Marco Agesic); CVE/CWE
- Estandares de buenas practicas: [ISO/IEC 27001, NIST CSF, OWASP, etc.].

4. Metodologia y herramientas

Como se desarrollo la actividad y con que herramientas (pip-audit (auditor gratuito de dependencias)).

5. Resultados y evidencias

Descripcion de lo obtenido, con referencia a las evidencias guardadas:

- Evidencia: EVI-...-01-pipaudit-antes.txt (auditoria inicial).
- Evidencia: EVI-...-02-pipaudit-despues.txt (tras actualizar).
- Evidencia: EVI-...-03-audit.json (reporte estructurado).
- Evidencia: EVI-...-04-cve-explicado.png (explicacion con IA).

6. Hallazgos, riesgos y recomendaciones

Presentacion de hallazgos ordenados por criticidad y las recomendaciones asociadas:

ID	Hallazgo	Criticidad	Recomendacion	Estado
H-01	[hallazgo]	[C/A/M/B]	[recomendacion]	[Abierto / Cerrado]

7. Gestion de incidentes (si aplica)

- Tipificacion del evento y datos involucrados.
- Notificacion a la Unidad Reguladora y de Control de Datos Personales si se vieron afectados datos personales.
- Lecciones aprendidas y mejoras al plan de respuesta.

8. Plan de mejora y continuidad

Mejora planificada	Responsable	Plazo	Prioridad
[mejora]	[cargo]	[dd/mm/aaaa]	[A/M/B]

9. Compromisos y firmas

El Banco Ficticio del Uruguay se compromete a [seguimiento, capacitacion, revision periodica] y presenta este informe para conocimiento de AGESIC.

Firma del RSI: _____ Sello del organismo: _____

11. Rubrica de evaluacion

Criterio	Logrado (3)	En desarrollo (2)	No logrado (1)
Instala y ejecuta pip-audit			
Interpreta CVEs y severidad (CVSS)			
Actualiza de forma segura y re-audita			
Genera y guarda el reporte JSON			
Propone una politica de gestion de dependencias			

12. Checklist de cierre

- ☐ Ejecuté pip-audit correctamente.
- ☐ Expliqué al menos 1 CVE.
- ☐ Actualicé y re-audité.

13. Reflexion sobre el uso de IA

¿Por que las dependencias son un "riesgo heredado" para un RSI? ¿Como se puede automatizar su revision en un equipo de desarrollo?